```
$ npm i @ai-sdk/google
```

The package `ai` is the core Vercel AI SDK, while `@ai-sdk/openai` is a separate package specifically designed to integrate with Google's services, and it follows the separation of concerns concept I explained earlier.

3 *Test the functionality*—After each change, you should check the functionality of the code. You can do this either by writing unit and integration tests or more directly by manually testing the site for errors. This will ensure you maintain good progress without losing track of what broke your application.

Now start the development server using the following command from the root folder of the project and visit `localhost:3000`:

```
$  npm run dev -w ch03/chat-vercel-ai
```

Now we will see that the functionality stays the same and nothing is broken.

That's it! You've successfully integrated the Vercel AI SDK into your app in three simple steps while learning exactly what has changed and why. Now let's start enhancing our existing functionality one feature at a time, beginning with supporting streaming responses. Things are about to get a bit more complex from here on.

3.2 Handling streaming responses with the Vercel AI SDK

While the text generation we've implemented is a great first step, modern AI applications demand a more dynamic experience. The solution is streaming responses, which provides real-time feedback and significantly improves the perceived performance of our application. At its core, streaming is a user experience solution to a technical problem. Large language models (LLMs) are computationally intensive and can be slow to produce a complete response. By delivering the content incrementally as it's generated, we mitigate the perceived wait time and create a more responsive and engaging interaction.

While the previous reasons highlight the benefits of streaming from a user experience perspective, it's important to understand the underlying technical rationale behind its adoption. At its core, the preference for streaming stems from the inherent limitations of LLMs. These models, which power AI applications, are computationally intensive and relatively slow in processing and generating responses. Figure 3.2 depicts the overall throughput of GPT-4 taken from https://artificialanalysis.ai, which is measured in tokens per second; it sits quite low at around 21 tokens per second.

This shows that even the top-tier LLMs really struggle to produce content quickly. Thus, streaming emerges as a solution to mitigate the effect of these limitations on the user experience. By delivering the generated content incrementally, we mitigate the bad user experience of having to wait for a response to be returned to the UI. Overall, it's an acceptable compromise for that kind of problem.